1. Tp sur transactions , spring-data (avec spring-boot)

Récupérer une copie de https://github.com/didier-tp/spring2026.git (via ***git clone*** ou via **code/download-zip** + extraction du zip dans c:\tp ou ailleurs)

1.1. (Tp facultatif) , nouveau projet via spring initializr

- Aller sur le site "spring initializr"
- Créer un nouveau projet ("demo" , "java/maven/java_17" , "jar") avec les dépendances fondamentales ("jpa" , "web" , …)
- extraire le contenu de demo.zip dans d:\tp ou ailleurs
- charger ce projet dans eclipse ou intelliJ
- ajouter un numéro de port dans application.properties
- ajouter une page index.html au sein de src/main/resources/static
- démarrer l'application (via le main()) et vérifier l'accès à la page d'accueil

NB: avec beaucoup de temps, on pourrait effectuer tous les Tps ultérieurs à partir de ce nouveau projet .

Pour gagner un peu de temps, les Tps ultérieurs seront effectués à partir d'un point de départ plus riche et bien structuré tp/debutAppliSpringWeb .

Néanmoins, pour les "débutants Spring" , enrichir petit à petit le point de départ "demo" peut être un exercice complémentaire intéressant si l'on souhaite bien comprendre toutes les configurations nécessaires au bon fonctionnement d'une application springBoot .

1.2. Chargement du projet springBoot et analyses/vérifications

L'objectif de cette seconde série de Tps est d'appréhender les fonctionnalités essentielles de Spring en mode "configuration moderne" (avec spring-boot et spring-data) .

Charger dans eclipse ou intelliJ le projet suivant (au format maven) :
tp/*debutAppliSpringWeb (de https://github.com/didier-tp/spring2026.git)*

NB: ce projet nécessite **java 17** comme version minimum du java .
Avec **intelliJ** , vérifier si besoin les réglages java/jdk via le menu **"file / project structure"**

- Analyser la structure de **pom.xml** (avec packaging="jar")
- Repérer les principales dépendances (spring-boot-starter-…)
- Lancer l'exécution de **tp.appliSpring.AppliSpringApplication.main()**
- Visualiser la page d'accueil au sein d'un navigateur (`http://localhost:8181/appliSpring`)
  `partie fonctionnant dès le début : compte1 au format json`
- Beaucoup de choses seront approfondies ultérieurement

1.3. Analyse de application.properties et variantes (profiles)

NB :

- Le fichier **application.properties** comporte les configurations essentielles (n° port, …)
- Le fichier application**-dev**.properties (associé au profil "dev") comporte une configuration vers une base de données "h2" (embedded database sans serveur) dont l'url est `jdbc:h2:~/mydbbank`
- Le fichier application**-dev2**.properties (associé au profil "dev2") comporte une configuration vers une base de données "mysql" (ou "mariaDB" compatible) dont l'url est jdbc:mysql://localhost:3306/mydbbank?createDatabaseIfNotExist=true
- **Les tables de cette base de données seront re-créées (à vide) à chaque redémarrage** (du main() ou des test avec le profil "dev" ou "dev2") du fait de la propriété `spring.jpa.hibernate.ddl-auto=create`
- D'autres profils (ex : "prod") comporte spring.jpa.hibernate.ddl-auto=**none**

1.4. Dao en mode JpaRepository

Analyser les portions de code suivantes :

tp.appliSpring.bank.persistence.entity.**CompteEntity**

tp.appliSpring.bank.persistence.repository.**CompteRepository**

Faire fonctionner le test suivant :

tp.appliSpring.core.dao.**TestCompteDao**

Phase1 du Tp :

Ajouter dans CompteRepository une méthode permettant de rechercher les comptes dont le solde est entre une valeur mini et une valeur maxi.

On proposera deux versions de cette méthode :

- une première respectant les convention de nommage des méthodes pour une requête automatique

- une seconde codée avec @Query() et JpaQL

On testera ceci en ajoutant une méthode de test au sein de TestCompteDao

Phase2 (facultative) du Tp :

- Ajouter dans OperationRepository une méthode permettant de rechercher les opérations rattachées à un numéro de compte précis et dont la date est entre une valeur mini et une valeur maxi (les paramètres dateMini , dateMaxi seront de type "Date")
- On testera ceci en ajoutant une méthode de test au sein de TestOperationDao (à coder à coté de TestCompteDao) avec par exemple SimpleDateFormat dateFormat = new SimpleDateFormat("yyyy-MM-dd"); et `dateFormat.parse("2025-02-12")`

Phase3 (facultative) du Tp :

- coder une variante (ex : TestOperationDaoSql) de la classe TestOperationDao avec l'utilisation de @Sql pour injecter des données en base en début de méthode de test

Phase4 (facultative) du Tp :

- expérimentations libres (sans casser ce qui fonctionne dès le départ)

1.5. Transactions sur virement bancaire

Analyser globalement la structure de code suivante :

- tp.appliSpring.bank.core.**model**.**Compte**
- tp.appliSpring.bank.core.service.**ServiceCompte**
- tp.appliSpring.bank.core.service.direct.***ServiceCompteDirectImpl***

Code important de la classe tp.appliSpring.bank.core.service.direct.***ServiceCompteDirectImpl*** :

```
// ...
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
// ...

@Service //classe de Service prise en charge par spring
//@Transactional
@Qualifier("direct")
@Primary
public class ServiceCompteDirectImpl …. implements ServiceCompte{

// ...

@Transactional(/*propagation = Propagation.REQUIRED*/) //REQUIRED par defaut
public void transfer(double montant, String numCptDeb, String numCptCred)
    throws BankException{
  try {
      // transaction globale initialisée dès le début de l'exécution de transferer
        Long numCptDebLong = Long.parseLong(numCptDeb);
        CompteEntity cptDeb = this.daoCompte.findById(numCptDebLong).get();
              //le dao exécute son code dans la grande transaction
              //commencée par le service sans la fermer et l'objet cptDeb remonte à l'état persistant
        cptDeb.setSolde(cptDeb.getSolde() - montant);
        this.daoCompte.save(cptDeb); //facultatif si @Transactional


        //idem pour compte à créditer
        Long numCptCredLong = Long.parseLong(numCptCred);
        CompteEntity cptCred= this.daoCompte.findById(numCptCredLong).get();
        cptCred.setSolde(cptCred.getSolde() + montant);
        this.daoCompte.save(cptCred) ; //facultatif si @Transactional

        //en fin de transaction réussie (sans exception) , toutes les modification effectuées
        //sur les objets à l'état persistant seront répercutées en base (.save() automatiques)
        } catch (Exception e) {
                throw new BankException("echec virement",e);
                //le rollback se fait de façon fiable car BankException hérite de RuntimeException
        }
  }
}
```

Au sein de ***src/test/java*** et du package ***tp.appliSpring.core.service*** , visualiser la classe de test ***TestServiceCompte*** avec la partie de code importante suivante :

```
// ...
@SpringBootTest(classes= {AppliSpringApplication.class})
@ActiveProfiles({  "dev" })
public class TestServiceCompte {

private static Logger logger = LoggerFactory.getLogger(TestServiceCompte.class);

@Autowired
private ServiceCompte serviceCompte; //à tester


@Test
public void testVirement() {
        Compte compteASauvegarde = this.serviceCompte.create(
                new Compte(null,"compteA",300.0));
        Compte compteBSauvegarde = this.serviceCompte.create(
                 new Compte(null,"compteB",100.0));
        String numCptA = compteASauvegarde.getNumero();
        String numCptB = compteBSauvegarde.getNumero();
        //remonter en memoire les anciens soldes des compte A et B avant virement
        //(+affichage console ou logger) :
        double soldeA_avant= compteASauvegarde.getSolde();
        double soldeB_avant = compteBSauvegarde.getSolde();
        logger.debug("avant bon virement, soldeA_avant="+soldeA_avant +
                        " et soldeB_avant=" + soldeB_avant);

        //effectuer un virement de 50 euros d'un compte A vers vers compte B
        this.serviceCompte.transfer(50.0, numCptA, numCptB);

        //remonter en memoire les nouveaux soldes des compte A et B apres virement
         // (+affichage console ou logger)
        Compte compteAReluApresVirement =
                     this.serviceCompte.searchById(numCptA);
        Compte compteBReluApresVirement =
                    this.serviceCompte.searchById(numCptB);
        double soldeA_apres = compteAReluApresVirement.getSolde();
        double soldeB_apres = compteBReluApresVirement.getSolde();
        logger.debug("apres bon virement, soldeA_apres="+soldeA_apres
                         + " et soldeB_apres=" + soldeB_apres);

        //verifier -50 et +50 sur les différences de soldes sur A et B :
        Assertions.assertEquals(soldeA_avant - 50, soldeA_apres,0.000001);
        Assertions.assertEquals(soldeB_avant + 50, soldeB_apres,0.000001);
}

//@Test
public void testMauvaisVirement() {
        /*   VARIANTE A CODER/COMPLETER EN TP
        COPIER/COLLER à ADPATER de  testVirement()
         AVEC
```

```
        try {
            this.serviceCompte.transfer(50.0, numCptA, "-1"); //erreur volontaire
        } catch (Exception e) {
                logger.error("echec normal du virement " + e.getMessage());
        }
        et
        //verifier -0 et +0 sur les différences de soldes sur A et B
        Assertions.assertEquals(soldeA_avant , soldeA_apres,0.000001);
        Assertions.assertEquals(soldeB_avant , soldeB_apres,0.000001);
         */
}
}
```

Série de tests à effectuer :

1. enlever ~~@Transactional~~ au dessus de la méthode **transfer**

2. *lancer le test* ***testVirement()*** et corriger les bugs si nécessaire
3. coder et lancer ***testMauvaisVirement***() . c'est normal si ça ne fonctionne pas bien sans l'ajout de @Transactional
4. replacer @Transactional au dessus de la méthode **transfer** et relancer le test ***testMauvaisVirement***() qui devrait normalement fonctionner .
5. Autres expérimentations libres (sans casser le code de départ)

1.6. Tp sur impact de @Transactional en mode lecture

1. Ajouter si besoin la méthode suivante dans l'interface du service "***ServiceOperation***" (du package tp.appliSpring.bank.core.service)
   public List<Operation> searchByCompte(String numCpt);

2. Ajouter si besoin ce code initial dans la classe d'implémentation ***ServiceOperationImpl***

```
//@Transactional(readOnly = true)
@Transactional()
public List<Operation> searchByCompte(String numCpt) {
  Long numCptLong = Long.parseLong(numCpt);
   //CompteEntity compteEntity = daoCompte.findWithOperations(numCptLong);
   CompteEntity compteEntity = daoCompte.findById(numCptLong).get();
   compteEntity.getOperations().size();// boucle interne automatique (lazy loading différé)
   return GenericMapper.MAPPER.map(compteEntity.getOperations(),Operation.class);
}
```

3. Au sein de la classe **TestServiceCompte** , ajouter si besoin
@Autowired private ServiceOperation serviceOperation;
et la nouvelle méthode de test suivante :

```
@Test
  public void testCompteAvecOperation(){
     Compte cptA = serviceCompte.create(new Compte(null,"compteA",100.0));
    serviceOperation.create(new Operation(null,"achat 1" , -5.0 , new Date()), cptA.getNumero());
    serviceOperation.create(new Operation(null,"achat 2" , -6.0 , new Date()), cptA.getNumero());
    Compte cptB = serviceCompte.create(new Compte(null,"compteB",200.0));

     Compte compteAReluAvecOp = serviceCompte.searchById(cptA.getNumero());
     assertTrue(compteAReluAvecOp.getLabel().equals("compteA"));
```

```
    // afficher les operations de compteAReluAvecOpavec le bon comportement de @Transactional :
    List<Operation> operations = serviceOperation.searchByCompte(cptA.getNumero());
    assertTrue(operations.size()==2);
    logger.debug("operations de compteAReluAvecOp="+operations);
}
```

et lancer ensuite plusieurs fois ce test en ajoutant ou retirant @Transactional au dessus du code de la méthode ***searchByCompte()*** *de la classe* ***ServiceOperationDirectImpl.***
On pourra tester plein d'autres variantes de ce code selon l'inspiration du moment ...